

DSA521S – DATA STRUCTURES AND ALGORITHMS 1 GROUP MINI-PROJECT 2026

PART A — DATA STRUCTURE DESIGN

A1. Waiting Line — Queue

Initial queue:

FRONT → Maria(12) → Tomas(5) → Ndapewa(8) → Simon(4) → Paula(7) → Marcus(10) → REAR

dequeue Maria:

FRONT → Tomas(5) → Ndapewa(8) → Simon(4) → Paula(7) → Marcus(10) → REAR

dequeue Tomas:

FRONT → Ndapewa(8) → Simon(4) → Paula(7) → Marcus(10) → REAR

dequeue Ndapewa:

FRONT → Simon(4) → Paula(7) → Marcus(10) → REAR

Queue pseudocode

```
enqueue(student)
    IF rear == capacity - 1
        DISPLAY "Queue is full"
        RETURN
    END IF
    IF front == -1
        front = 0
    END IF
    rear = rear + 1
    queue[rear] = student
END

dequeue()
    IF isEmpty()
        DISPLAY "Queue is empty"
        RETURN null
    END IF
    student = queue[front]
    front = front + 1
    IF front > rear
        front = -1
        rear = -1
    END IF
    RETURN student
END
```

```
peek()
    IF isEmpty() RETURN null
    RETURN queue[front]
```

```
isEmpty()
    RETURN front == -1
```

```
displayQueue()
```

```

IF isEmpty() DISPLAY "Queue is empty"
ELSE
    FOR i = front TO rear
        DISPLAY queue[i]
    END FOR
END IF

```

A queue is appropriate because students are normally assisted in the order in which they arrive: FIFO (First In, First Out).

A2. Student Service Records — Singly Linked List

Node: [StudentNo | Name | ServiceType | Time | next]

Insertion Before:

HEAD → [Maria | Registration | 12] → [Tomas | Student Card | 5] → [Simon | Documents | 4] → NULL

Insert Ndapewa at position 3:

HEAD → [Maria] → [Tomas] → [Ndapewa | Fees | 8] → [Simon] → NULL

Delete Tomas:

HEAD → [Maria] → [Ndapewa] → [Simon] → NULL

Required operations

insertNode(student, position)

newNode = create node(student)

IF position == 1

newNode.next = HEAD

HEAD = newNode

RETURN

END IF

current = HEAD

FOR i = 1 TO position - 2

IF current == NULL RETURN

current = current.next

END FOR

newNode.next = current.next

current.next = newNode

END

deleteNode(studentNo)

IF HEAD == NULL RETURN

IF HEAD.studentNo == studentNo

HEAD = HEAD.next

RETURN

END IF

current = HEAD

WHILE current.next != NULL AND current.next.studentNo != studentNo

current = current.next

END WHILE

IF current.next != NULL

current.next = current.next.next

END IF

END

searchNode(studentNo)

current = HEAD

WHILE current != NULL

IF current.studentNo == studentNo RETURN current

current = current.next

```

END WHILE
RETURN NULL
traverseList()
current = HEAD
WHILE current != NULL
    DISPLAY current data
    current = current.next
END WHILE

```

A3. Postfix Expression Evaluation — Stack

Token	Operation	Stack after operation
5	push(5)	[5]
3	Push(3)	[5,3]
+	Pop 3, pop 5; 5+3=8; push(8)	[8]
2	push(2)	[8,2]
*	Pop 2, pop 8; 8x2=16; push(16)	[16]

A4. Daily Statistics — Array

Service-time array: [12, 5, 8, 4, 9, 11, 15, 7]

Total students served = 8; total service time = 71 minutes; average = 8.88 minutes; highest = 15; lowest = 4; services longer than 10 minutes = 3.

```

total = 0
highest = serviceTimes[0]
lowest = serviceTimes[0]
longerThan10 = 0
FOR i = 0 TO length - 1
    total = total + serviceTimes[i]
    IF serviceTimes[i] > highest
        highest = serviceTimes[i]
    END IF
    IF serviceTimes[i] < lowest
        lowest = serviceTimes[i]
    END IF
    IF serviceTimes[i] > 10
        longerThan10 = longerThan10 + 1
    END IF
END FOR
average = total / length

```

A5. Data-Structure Justification

Structure

Array = Service times are stored in indexed positions and can be traversed efficiently for daily statistics.

Singly linked list = Service times are stored in indexed positions and can be traversed efficiently for daily statistics.

Stack = Service times are stored in indexed positions and can be traversed efficiently for daily statistics.

Queue = Service times are stored in indexed positions and can be traversed efficiently for daily statistics.

PART B — SORTING ALGORITHM CHALLENGE

Array: [17, 5, 23, 8, 3, 11, 20, 6, 9]

Selection sort, first 3 passes

Pass	Array State
1	[3, 5, 23, 8, 14, 17, 11, 20, 6, 9]
2	[3, 5, 6, 8, 14, 17, 11, 20, 23, 9]
3	[3, 5, 6, 8, 9, 17, 11, 20, 23, 16]

Final sorted array: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

```
selectionSort(A)
  FOR i = 0 TO n-2
    min = i
    FOR j = i+1 TO n-1
      IF A[j] < A[min]
        min = j
      END IF
    END FOR
    swap A[i] and A[min]
  END FOR
```

B2. Insertion Sort — first three passes

Pass	Array State
1	[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]
2	[5, 17, 23, 8, 14, 3, 11, 20, 6, 9]
3	[5, 8, 17, 23, 14, 3, 11, 20, 6, 9]

Final inserted array: [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

```
insertionSort(A)
  FOR i = 1 TO n-1
    key = A[i]
    j = i-1
    WHILE j >= 0 AND A[j] > key
      A[j+1] = A[j]
      j = j-1
    END WHILE
    A[j+1] = key
  END FOR
```

B3. Merge Sort — divide and merge

[17,5,23,8,14,3,11,20,6,9]
/ \
[17,5,23,8,14] [3,11,20,6,9]
/ \
[17,5] [23,8,14] [3,11] [20,6,9]
/\ / \
[17][5] [23] [8,14] [3][11] [20] [6,9]
... continue splitting until single values ...

Base case: a subarray containing one element is already sorted.

Merge the single-element arrays:

[5, 17] ; [8, 14, 23] ; [3, 11] ; [6, 9]
→ [5, 8, 14, 17, 23] and [3, 6, 9, 11, 20]
→ [3, 5, 6, 8, 9, 11, 14, 17, 20, 23]

```
mergeSort(A)
  IF length(A) <= 1
    RETURN
  END IF
  split A into left and right halves
  mergeSort(left)
  mergeSort(right)
  merge left and right into sorted A
```

B4. Quick Sort — partition stages

Pivot rule used here: choose the last element of the current partition as the pivot (Lomuto partition scheme).

Initial: [17,5,23,8,14,3,11,20,6,9]
Pivot = 9
Left partition: [5,8,3,6]
Pivot: [9]
Right partition: [17,23,14,11,20]
First right-side partition: [17,23,14,11,20], pivot = 20
Left partition: [17,14,11]
Pivot: [20]
Right partition: [23]
Continue recursively until every partition has size 0 or 1.

PART C — ALGORITHM EXPERIMENT

Algorithm	20 comps/time(ns)	50 comps/time(ns)	100 comps/time(ns)	500 comps/time(ns)
Selection Sort	Run program	Run program	Run program	Run program
Insertion Sort	Run program	Run program	Run program	Run program
Merge Sort	Run program	Run program	Run program	Run program
Quick Sort	Run program	Run program	Run program	Run program

Analysis answers

1. Which algorithm performed the fewest comparisons?

All the algorithms performed the fewest comparisons.

2. Which algorithm performed the most comparisons?

All the algorithms performed the most comparisons.

3. Which algorithm performed best on the almost-sorted array?

All the algorithms performed the almost-sorted array.

4. Which algorithm appeared to scale best as input became larger?

Merge Sort and a well-behaved Quick Sort generally scale much better than quadratic algorithms.

5. Did results generally agree with theoretical complexity?

They should broadly reflect Selection/Insertion Sort $O(n^2)$ in general, versus Merge Sort $O(n \log n)$ and average Quick Sort $O(n \log n)$. Actual timing can vary because of JVM and hardware effects.

6. Is execution time alone enough?

No. Time is affected by hardware, JVM warm-up, background processes and implementation details. Comparisons, theoretical complexity, memory usage, input characteristics and correctness should also be considered.

PART D — INTEGRATED SERVICE-CENTRE SYSTEM

Recommended menu

=====

CAMPUS SERVICE CENTRE

=====

1. Add student to waiting queue
2. Serve next student
3. Display waiting students
4. Add student service record
5. Display student service records
6. Search for student record
7. Remove student record
8. Display daily statistics
9. Sort service times
10. Run sorting experiment
11. Exit

Select option:

Menu mapping

Option	Required implementation
1	Queue — enqueue
2	Queue — dequeue
3	Queue — display/traversal
4	Singly Linked List — insertion
5	Singly Linked List — traversal
6	Singly Linked List — search
7	Singly Linked List — deletion
8	Array processing
9	Sorting algorithm(s)
10	Sorting experiment
11	Exit

PART E — PSEUDOCODE AND ALGORITHM REPRESENTATION

QUEUE

enqueue(student):

- if full -> report full
- else place student at rear

dequeue():

- if empty -> report empty
- else remove and return student at front

STACK

push(x):

- if top == size-1 -> overflow
- else top++; stack[top]=x

pop():

- if top == -1 -> underflow
- else return stack[top--]

POSTFIX:

for each token:

- number -> push

- operator -> b=pop, a=pop, push(a operator b)

return pop

LINKED LIST

insertNode(student, position):

- create node

- if position=1 -> new.next=head; head=new

- else move to node before position

- link new node into list

deleteNode(studentNo):

- find previous node

- previous.next = previous.next.next

searchNode(studentNo):

- traverse until matching studentNo or NULL

traverseList():

- visit every node from HEAD to NULL

SORTING

selectionSort:

- for each position, find smallest remaining value and swap

insertionSort:

- take current value and shift larger values right

mergeSort:

- split recursively until one-element arrays

- merge sorted halves

quickSort:

- select pivot

- partition into values smaller/larger than pivot

- recursively sort both partitions

Reference Java implementation structure

```
class Student {  
    int studentNo;  
    String name, serviceType;  
    int serviceTime;  
}  
class Node {  
    Student data;  
    Node next;  
}
```

```

class StudentList {
    Node head;
    void insertStudent(Student s, int position) { /* implement links */ }
    void deleteStudent(int studentNo) { /* implement links */ }
    Student searchStudent(int studentNo) { /* traverse */ return null; }
    void displayStudents() { /* traverse */ }
}

class StudentQueue {
    Student[] queue;
    int front = -1, rear = -1;
    void enqueue(Student s) { /* FIFO insertion */ }
    Student dequeue() { /* FIFO removal */ return null; }
    Student peek() { return null; }
    boolean isEmpty() { return front == -1; }
    void displayQueue() { /* traverse queue */ }
}

class IntStack {
    int[] stack;
    int top = -1;
    void push(int x) { /* overflow check + push */ }
    int pop() { /* underflow check + pop */ return 0; }
    int peek() { return stack[top]; }
}

class Sorts {
    static void selectionSort(int[] a) { /* count comparisons/swaps */ }
    static void insertionSort(int[] a) { /* count comparisons/shifts */ }
    static void mergeSort(int[] a) { /* recursive merge sort */ }
    static void quickSort(int[] a) { /* recursive quick sort */ }
}

```